BendTT: An Affine Dependent Type Theory

Victor Taelin, Lorenzo W Battistela, Paulo J Cavalcanti
Nicolas Abril, Vitor Chiarelli Neves, Vanessa Ostroski
Higher Order Company
Rio de Janeiro, Brazil
taelin@higherorderco.com

AI Disclosure. Bend and BendTT were designed by the human author. This paper was written by Claude Opus 5.5 from the author's code and design choices, and reviewed by the author. The Lean mechanization was human-specified, AI-proven, and verified by a computer. Human paper soon™.

Abstract

BendTT is the core type theory of Bend, a programming language in which programs and their proofs are written together. It has Type : Type, impredicative function types, and datatypes whose fields may be functions over the datatype itself. Proof assistants such as Coq, Agda and Lean forbid Type : Type and such datatypes, because each one leads to a proof of false. BendTT allows both and is consistent. The reason is that the code that runs is affine. It may copy first-order data, and it may not copy a function. Girard's paradox and Curry's paradox both copy a function, so both fail to check. Types never run, so they are free to use a variable many times. We prove confluence, subject reduction, progress, termination of running code and consistency in a single Lean 4 file. The theorems are about the checker in that file, and Bend can run that checker on its programs.

1 Introduction

Proof assistants based on dependent types, like Coq, Agda and Lean, restrict two things. First, they stack their types in a hierarchy of universes, because a type of all types, Type : Type, makes the logic inconsistent [23, 28]. Second, they reject a datatype that occurs to the left of an arrow in its own constructors, because such a type lets a program loop without recursion [16, 34]. In a logic, a program that loops can have any type, including the empty type, so it proves anything. Both restrictions cost the user something. Universe levels must be managed, and some useful types, like terms in higher-order abstract syntax, cannot be declared.

BendTT is the core type theory of the Bend programming language. It has no universe levels and no positivity check. Here is the usual first step towards a contradiction, in Bend:

```
type R is Type:
  Fold{f: R -> Empty}

def app(r: R) -> Empty:
  match r:
    case Fold{f}:
      f(Fold{f})
```

```
Error:
- expected : f
- observed : f (consumed more than once)
Location: app
```

The type `R` occurs to the left of an arrow in its own field, and Bend accepts it. If Bend also accepted `app`, then `app(Fold{app})` would have type `Empty`, and it would reduce to itself forever. This is Curry's paradox [18], in the form that positivity checks exist to stop. Bend rejects `app` for another reason. Its body uses `f` twice, once to call it and once to rebuild the argument. In Bend, a variable of a function type may be used at most once.

That is the main idea of this paper. The classical paradoxes of Type : Type and of negative datatypes copy a function (Section 3). So a language whose running code cannot copy functions can keep both features. The catch is that types must mention variables many times. The statement `{add(a, b) == add(b, a) : Nat}` mentions `a` and `b` twice each. So BendTT splits every term in two parts. The *live* part is the code that runs. The *dead* part is the code that is only type-checked: types, annotations, and arguments that are erased before the program runs. Live code must follow three rules. Dead code need not follow any of them.

1. A variable is used at most once. This is *affinity*.
2. A variable may be used more than once only if its type has the kind Data. At run time, a value of a Data type is made of labels, pairs and equality proofs, so it holds no function.
3. A definition may call earlier definitions. It may call itself only on smaller arguments.

With these rules, the proof of consistency is short. Suppose that a definition k had the empty type. The name k is live code, so its evaluation ends. Evaluation keeps the type and does not get stuck, so it ends in a value of the empty type. There is no such value. The termination argument counts calls and ignores types. It works because affine code cannot copy work that is still to be done. No step of the proof asks a type to normalize, so Type : Type does no harm. The dead part of BendTT can even hold all of Girard's paradox, and the checker accepts it there (Section 3.2).

2 A Tour of Bend

Every Bend example in this paper imports Bend's base library, which defines `Nat`, `List` and `Empty`. We ran each example through the Bend checker, and each accepted example also through the BendTT kernel (Section 8). We show each error message up to its `Location` line.

2.1 Programs and Proofs

Here is addition on natural numbers:

```
def add(a: Nat, b: Nat) -> Nat:
  match a:
    case 0n:
      b
    case 1n+p:
      1n+add(p, b)
```

The patterns `0n` and `1n+p` stand for `Zero{}` and `Succ{p}`. A proof is a program too. This one shows that `add` is commutative:

```
law comm:
  for +a: Nat
  for +b: Nat
  {add(a, b) == add(b, a) : Nat}

def comm(a, b):
  match a:
    case 0n:
      zero(b)
    case 1n+p:
      %succ(b, p) : {1n+add(p, b) == _ : Nat}
      %comm(p, b) : {1n+add(p, b) == 1n+_ : Nat}
      {==}
```

A `law` states a type, and the `def` with the same name is a term of that type. The type `{x == y : T}` says that `x` and `y` are equal values of type `T`. Its proof `{==}` checks when both sides compute to the same term. The lemmas `zero(a)`, of type `{a == add(a, 0n) : Nat}`, and `succ(a, b)`, of type `{1n+add(a, b) == add(a, 1n+b) : Nat}`, have proofs of the same shape.

The proof is by induction, and induction is recursion. The `match` splits the goal in two cases, and in each case the checker replaces `a` by its pattern. In the first case, `add(0n, b)` computes to `b`, and `zero(b)` proves the goal. In the second case, the recursive call `comm(p, b)` is the induction hypothesis. Bend accepts it because `p` is a piece of `a`. A rewrite `%e : P` uses an equation `e : {x == y : T}`. The motive `P` is a type with a hole `_`. With `y` in the hole, `P` must be the goal. With `x` in the hole, `P` is the new goal. Bend has no tactics, and a proof is an ordinary term.

2.2 Quantities

Every binder has a *quantity*. A plain binder, like `b` in `add`, is *affine*. The running code may use it at most once. A binder marked `-`, like `-A: Type`, is *erased*. It may appear only in dead code, and the compiler removes it. A binder marked `+`, like `+a` in `comm`, is *copyable*. The running code may use it any number of times. In the second case of `comm`, the proof uses `p` twice and `b` twice, so the law marks `a` and `b` with `+`. A match on a copyable value gives copyable fields.

2.3 Kinds

Every type has a *kind* `Kind(q)`, for a quantity `q`: Type is `Kind(&1)` and Data is `Kind(&2)`. A copyable binder needs a type of kind Data. A datatype declares its kind, and the checker verifies the kind at each field. A Data type may only have Data fields. `Nat` is Data. A function type is always Type, so a function cannot be copyable:

```
law dupf:
  for +f: Nat -> Nat
  Nat
```

```
Error:
- expected : Data
- observed : Type
Location: dupf
```

For the same reason, the checker rejects the type `R` of the introduction if it is declared `is Data`. It gives the same error, at the field `f`.

2.4 Dead Code

These rules apply to live code only. A law is a type, so it is dead, and it may mention a function as often as it needs. This law uses `f` four times, and `{==}` proves it:

```
law twice:
  for f: Nat -> Nat
  for x: Nat
  {f(f(x)) == f(f(x)) : Nat}

def twice(f, x):
  {==}
```

Type : Type is also free to use. A function may return a type, and a polymorphic function may be applied to its own type:

```
def Pred(A: Type) -> Type:
  A -> Type

def id(-A: Type, x: A) -> A:
  x

law idid:
  @-A: Type -> @_: A -> A

def idid():
  id(@-A: Type -> @_: A -> A, id)
```

The type `@-A: Type -> @_: A -> A` quantifies over all types, itself included.

3 Why the Paradoxes Copy

This section looks at the two classical paradoxes. Each one copies a function, and BendTT rejects that copy. We do not claim that every possible paradox must copy a function. The consistency theorem of Section 6 covers every term, whatever its shape.

3.1 Curry's Paradox

The untyped term $(\lambda x.x\ x)(\lambda x.x\ x)$ reduces to itself. Simple types reject it, because $x\ x$ needs x to be a function whose domain is the type of x itself. The negative type `R` of the introduction provides such a type, and `app(Fold{app})` is the self-application. No definition calls itself, so a termination checker that inspects recursive calls finds nothing to reject. Coq and Agda reject the declaration of `R`. BendTT

```
P : ∀S : *1 -> *1 =
  λS => ∀x : S -> *1

Bot : *1 =
  ∀A : *1 -> A

Not : ∀A : *1 -> *1 =
  λA => ∀h : A -> Bot

U : *1 =
  ∀X : *1 -> ∀f : (∀g : (P (P X)) -> X) -> (P (P X))

girard : <()> =
  !-tau = {λt => λX => λf => λp => (t λx => (p (f (x X f)))) : ∀t : (P (P U)) -> U};
  !-sigma = {λs => (s U tau) : ∀s : U -> (P (P U))};
  !-Delta = {λy => (Not ∀p : (P U) -> ∀h : (sigma y p) -> (p (tau (sigma y)))) : (P U)};
  !-Omega = {(tau λp => ∀x : U -> ∀h : (sigma x p) -> (p x)) : U};
  !-lemma1 = {λp => λH1 => (H1 Omega λx => (H1 (tau (sigma x))))
    : ∀p : (P U) -> ∀H1 : (∀x : U -> ∀h : (sigma x p) -> (p x)) -> (p Omega)};
  !-lemma2 = {λH0 => (H0 Delta (λx => λH2 => λH3 => (H3 Delta H2 λp => (H3 λy => (p (tau (sigma y))))))
                               λp => (H0 λy => (p (tau (sigma y)))))
    : (Not ∀p : (P U) -> ∀H1 : (∀x : U -> ∀h : (sigma x p) -> (p x)) -> (p Omega))};
  !-false = {(lemma2 lemma1 <>) : <>};
  ()
```

Figure 1. Hurkens' paradox in BendTT. `*1` is Type, `<>` is the empty type, and `<()>` is the unit type. Each `!-` binds an erased let. The kernel prints `ALL PROOFS CHECK`. When the lets are live (`!` in place of `!-`) and `girard` returns `false` at type `<>`, the kernel prints `SOME PROOFS FAIL` and `affine live code, calls that descend`.

accepts `R` and rejects the second use of `f`. The field `f` cannot be made copyable, because its type is not Data.

3.2 Girard's Paradox

Girard showed that Type : Type makes Martin-Löf's first type theory [32] inconsistent [23], and Hurkens gave a short proof [28]. Let $\mathcal{P}X = X \to \mathsf{Type}$. Hurkens' proof uses the type $U = \forall X : \mathsf{Type}.(\mathcal{P}\mathcal{P}X \to X) \to \mathcal{P}\mathcal{P}X$ and the map $\tau : \mathcal{P}\mathcal{P}U \to U$ defined by

$$\tau\ t = \lambda X.\lambda f.\lambda p.\ t\ (\lambda x.\ p\ (f\ (x\ X\ f))).$$

The variable f occurs twice in the body of τ. It is applied, and it is passed to x. Written as a BendTT definition, τ fails the live check for this reason.

Figure 1 shows the whole proof in BendTT's text syntax (Section 4). This time every step is an erased let, `!-x = v; t`, whose value `v` is dead. The kernel type-checks every step, and it accepts the file. So the definition `girard` holds a closed term, `false`, of the empty type `<>`. As a logic, the dead part of BendTT is inconsistent. To make sure that the kernel checks dead code, we broke one step of `lemma1`, and the kernel rejected the file with a type mismatch.

This does no harm, because live code cannot reach `false`. The let binds it at quantity 0, so a live use of it breaks affinity. A match on it fails too, because the typing rules require a match to take a live argument (Section 4.6). The same holds in Bend:

```
def bad(-e: Empty) -> Nat:
  match e:
```

```
Error:
- message  : a live scrutinee (a - scrutinee
matches only in a dead region)
Location: bad
```

A dead term may also loop. The term `false` is closed and has the empty type, so by Theorems 2 and 3 and Lemma 4 of Section 6, its evaluation never ends. This does not contradict Theorem 5, because `false` fails the live check. Evaluation does not enter dead code, so this loop does not run.

4 The Calculus

BendTT is the small core language that Bend compiles to. It has no built-in datatypes and no fixed-point operator. A datatype is a pair of a label and the fields that the label needs. Recursion goes through the names of definitions.

4.1 Terms

Figure 2 lists the terms. We use two notations for them. The kernel reads a text syntax, which we use in examples. The rules use math notation. Every binder, application and pair carries a quantity 0, 1 or 2, which the text writes as `-`, nothing, or `+`. These quantities are fixed. A kind carries a quantity too, but there it is a term: a label of the enumeration $\mathsf{Q} = \langle Q_0, Q_1, Q_2 \rangle$, or a variable, a call or a meet that computes one. A *book* is an ordered list of definitions `k : T = t`, where `t` is closed. A name `k` in a term refers to a definition of the book.

Text	Math	Term
`x , k`	x, k	variable, name
`{t : T}`	$(t : T)$	annotation
`!q x = v; t`	$\mathsf{let}^q x = v; t$	let
`*(g)`	$\star\,(g)$	kind of quantity g
`*1 , *2`	$\mathsf{Type}, \mathsf{Data}$	$\star\,(Q_1), \star\,(Q_2)$
`(g <&> h)`	$g \sqcap h$	meet of quantities
`∀q x : A -> B`	$\forall^q (x : A).B$	function type
`λq x => t`	$\lambda^q x.t$	function
`(f q a)`	$f \circ^q a$	application
`Σq x : A -> B`	$\Sigma^q (x : A).B$	pair type
`(q a, b)`	$(a,^q b)$	pair
`<k1, k2> , .k`	$\langle k_1, k_2 \rangle, k$	enum, label
`λ{(,): h}`	$\lambda\{(,) : h\}$	pair match
`λ{.k: h; m}`	$\lambda\{k : h; m\}$	label match
`λ{}`	$\lambda\{\}$	empty match
`{a == b : A}`	$a =_A b$	equality
`{==}`	rfl	reflexivity
`%e : x, h => P; f`	$e \triangleright_{x,h.P} f$	rewrite (J)

Figure 2. The terms of BendTT, in text syntax and in math notation. The quantity q is 0 (erased), 1 (affine) or 2 (copyable).

4.2 Datatypes

Here is `Nat`, as Bend compiles it (we renamed the binders):

```
Nat.arms : ∀t : <Zero, Succ> -> *2 =
  λ{.Zero: <()>;
  λ{.Succ: Σp : Nat -> <()>;
  λ{}}}

Nat : *2 =
  Σt : <Zero, Succ> -> (Nat.arms t)
```

A `Nat` is a pair. Its first field is a label, `.Zero` or `.Succ`. The type of its second field depends on that label, and `Nat.arms` gives it. `Zero` has no fields, so it gets the unit type `<()>`. `Succ` has one field of type `Nat`, followed by a unit. The number 2 is `(.Succ, ((.Succ, ((.Zero, ()), ())), ()))`.

The names `Nat` and `Nat.arms` refer to each other, with no fixed-point operator. Both references are in types, which are dead. The live check ignores them, so they need no order and no positivity condition.

4.3 Kinds

The kind of a type says whether its values may be copied. A kind is $\star\,(g)$ for a quantity $g : \mathsf{Q}$, and a type of kind $\star\,(g)$ is Data exactly where g reduces to Q_2. Function types have kind Type. Enumerations and equality types have kind Data. A pair type $\Sigma^q (x : A).B$ has any kind $K = \star\,(g)$ that B has, and A must have the kind $\mathsf{kind}(q, K)$:

$$\mathsf{kind}(0, K) = \mathsf{Type}, \quad \mathsf{kind}(1, K) = K, \quad \mathsf{kind}(2, K) = \mathsf{Data}.$$

An erased field may have any type, since it is gone at run time. An affine field of a Data pair must be Data, and a copyable field is always Data. So the live part of a value of a Data type holds only labels, pairs and rfl. In the same way, the domain of a function type of quantity 2 must be Data.

A kind $\star\,(g)$ may be used where $\star\,(h)$ is expected when g is Q_2 wherever h is. So Data fits every kind, and every kind fits Type and $\star\,(Q_0)$. The meet $g \sqcap h$ is the least of two quantities. It reduces as Bend's `<&>` does: Q_2 is its identity and Q_0 absorbs it, on either side, even when the other side is stuck. So Bend's `Either<a, b, A, B>` has the kind $\star\,(a \sqcap b)$, and it is Data where both sides are. Function types fit by their parts, as in Bend: $\forall^q (x : A).B$ fits $\forall^q (x : C).D$ when C fits A and B fits D. So a `Nat -> Data` fits where a `Nat -> Type` is expected, and a `Type -> Nat` where a `Data -> Nat` is.

4.4 Matches and Case Trees

A match is a function that consumes its argument. The match $\lambda\{(,) : h\}$ takes a pair (a, b) and applies h to a and b. The match $\lambda\{k : h; m\}$ takes a label. On k it returns h. On any other label it passes the label to m, whose domain lacks k. The match $\lambda\{\}$ takes a value of the empty enumeration, and there is none. So a chain of label matches that ends in $\lambda\{\}$ covers every case. Here is `add`, as Bend compiles it (again with renamed binders):

```
add : ∀a : Nat -> ∀b : Nat -> Nat =
  λ{(,):
    λ{.Zero: λ{(): λb => b; λ{}};
    λ{.Succ: λ{(,): λp => λ{(): λb =>
      (.Succ, ((add p b), ())); λ{}}};
    λ{}}}}
```

The parameter `b` is bound inside each case. The reason is that the live check adds the uses of a variable in the two arms of a match (Section 5.1).

A definition calls itself, or another definition, by name. The *case tree* of a definition is its leading lambdas and matches. A call $k\ a_1 ... a_n$ unfolds only when its arguments lead through the whole case tree to a leaf. Otherwise the call stays as it is. For example, `(add p b)` with a variable `p` does not unfold, since the first match needs a pair. This rule has two uses. In proofs, the goal shows `add(p, b)` and not a half-evaluated match, so the induction hypothesis can match it. In the termination proof, a call that unfolds becomes one leaf in one step, and the calls in that leaf can be compared with the call that unfolded.

Here is Curry's paradox of the introduction, written directly in BendTT:

```
R.arms : ∀t : <Fold> -> *1 =
  λ{.Fold: Σf : ∀r : R -> <> -> <()>; λ{}}

R : *1 =
  Σt : <Fold> -> (R.arms t)

app : ∀r : R -> <> =
  λ{(,): λ{.Fold: λ{(,): λf => λ{():
    (f (.Fold, (f, ()))); λ{}}}; λ{}}}
```

```
In app:
affine live code, calls that descend
```

The kernel accepts `R` and rejects `app`, whose body uses `f` twice.

4.5 Equality

The type $a =_A b$ has kind Data. Its only value is rfl, which proves $a =_A b$ when a and b are convertible. The rewrite

$$\frac{\Gamma \vdash g : \mathsf{Q}}{\Gamma \vdash \star\,(g) : \mathsf{Type}}\ \textsc{sort} \qquad \frac{\Gamma \vdash g : \mathsf{Q} \quad \Gamma \vdash h : \mathsf{Q}}{\Gamma \vdash g \sqcap h : \mathsf{Q}}\ \textsc{meet} \qquad \frac{}{\Gamma \vdash \langle k_1, \ldots, k_n \rangle : \mathsf{Data}}\ \textsc{enum} \qquad \frac{\Gamma \vdash t : U \quad U \leq T}{\Gamma \vdash t : T}\ \textsc{conv} \qquad \frac{a \equiv b}{\Gamma \vdash \mathsf{rfl} : a =_A b}\ \textsc{refl}$$

$$\frac{\Gamma \vdash A : \mathsf{kind}(q, \mathsf{Type}) \quad \Gamma, x : A \vdash B : \mathsf{Type}}{\Gamma \vdash \forall^q (x : A).B : \mathsf{Type}}\ \textsc{pi} \qquad \frac{\Gamma \vdash A : \mathsf{kind}(q, K) \quad \Gamma, x : A \vdash B : K}{\Gamma \vdash \Sigma^q (x : A).B : K}\ \textsc{sigma}$$

$$\frac{\mathsf{live}(p) = \mathsf{live}(q) \quad q = 2 \Rightarrow \Gamma \vdash A : \mathsf{Data} \quad \Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda^q x.t : \forall^p (x : A).B}\ \textsc{lam} \qquad \frac{\Gamma \vdash f : \forall^q (x : A).B \quad \Gamma \vdash a : A}{\Gamma \vdash f \circ^q a : B[a/x]}\ \textsc{app}$$

$$\frac{\Gamma \vdash v : V \quad q = 2 \Rightarrow \Gamma \vdash V : \mathsf{Data} \quad \Gamma \vdash t[v/x] : T}{\Gamma \vdash \mathsf{let}^q x = v; t : T}\ \textsc{let} \qquad \frac{\mathsf{live}(q)}{\Gamma \vdash \lambda\{\} : \forall^q (s : \langle\rangle).P}\ \textsc{absurd}$$

$$\frac{\mathsf{live}(q) \quad \Gamma \vdash h : \forall^{\mathsf{fld}(r,q)} (x : A).\forall^q (y : B).P[(x,^r y)/s]}{\Gamma \vdash \lambda\{(,) : h\} : \forall^q (s : \Sigma^r (x : A).B).P}\ \textsc{split}$$

$$\frac{\mathsf{live}(q) \quad k \in L \quad \Gamma \vdash h : P[k/s] \quad \Gamma \vdash m : \forall^q (s : \langle L \setminus k \rangle).P}{\Gamma \vdash \lambda\{k : h; m\} : \forall^q (s : \langle L \rangle).P}\ \textsc{case}$$

$$\frac{\Gamma \vdash e : a =_A b \quad \Gamma, x : A, h : a =_A x \vdash P : \mathsf{Type} \quad \Gamma \vdash f : P[a, \mathsf{rfl}\ /x, h] \quad P[b, e/x, h] \leq T}{\Gamma \vdash e \triangleright_{x,h.P} f : T}\ \textsc{j}$$

Figure 3. Selected typing rules. $U \leq T$ holds when U and T are convertible, when they are kinds $\star\,(g)$ and $\star\,(h)$ and g is Q_2 wherever h is, under every substitution, when both are enumerations and each label of U is in T, when they are $\forall^q (x : A).B$ and $\forall^q (x : C).D$ with $C \leq A$ and $B \leq D$, or through a type between them. The checker decides the kind case by rules: Q_2 is the top, Q_0 and Q_1 the bottom, a meet on the left needs both sides, and a meet on the right needs either. The rules for variables, names, annotations, pairs, labels and the types of pairs and equalities are standard.

$e \triangleright_{x,h.P} f$ is the J eliminator of Martin-Löf type theory, with an explicit motive P. The motive may depend on both the right endpoint x and the proof $h : a =_A x$. The proof e is live. At run time it evaluates to rfl, and then the rewrite steps to f. If e could be dead, a dead proof of a false equation could change the type of live code, and evaluation could get stuck.

4.6 Typing

Figure 3 gives the typing rules. Here K is a kind $\star\,(g)$, and $\mathsf{live}(q)$ holds when q is 1 or 2. Most rules are standard. We explain the unusual ones.

The rule SORT gives every kind the kind Type, so Type : Type and Data : Type. There is one level, as in the `idid` example of Section 2.

The rules PI and SIGMA assign kinds. A function type always has kind Type. No rule gives it kind Data, and confluence (Section 6) shows that Type and Data are not convertible, since Q_1 and Q_2 are distinct labels. So no function is ever bound at quantity 2.

In the rule LAM, a lambda may bind its variable at quantity 2 under a function type of quantity 1, if the domain is Data. The caller passes one value, and the callee copies it. In APP, the application carries the quantity of the binder. So the live check can tell a dead argument from a live one without reading types.

The rules SPLIT, CASE and ABSURD give matches their dependent types. The motive P depends on the argument s, and each case sees P at its pattern. A pair match passes its first field at quantity $\mathsf{fld}(r, q)$, which is 0 when $r = 0$, q when $r = 1$, and 2 when $r = 2$. So a match on a copyable pair gives copyable fields. All three rules require a live quantity q, so a match cannot take a dead argument.

The rule J checks the body f with a and rfl in the motive, and it gives the rewrite the motive at b and e.

The checker is bidirectional [21]. When it checks a match applied to a variable x against a goal T, it takes the motive to be T with x abstracted. So each case sees the goal with x replaced by its pattern, as in the proof of `comm`. The kernel has no unification, so Bend matches only on variables.

With Type : Type, conversion is undecidable [26, 35], and a type may loop. The checker runs with a step budget, and it rejects a book when the budget runs out. A loop in a type can make the checker reject a correct book. It cannot make the checker accept a wrong one.

5 The Live Check

The typing rules do not count uses. A second pass, the *live check*, counts them, and it reads no types. These positions are *dead*: the type of an annotation, both parts of a $\forall$ or Σ type, the motive of a rewrite, the argument of $f \circ^0 a$, the first field of $(a,^0 b)$, and the value of $\mathsf{let}^0 x = v; t$. All other positions are *live*, and the body of a definition is live. The live check has two rules, affinity and descent.

5.1 Affinity

A variable of quantity 0 has no live use. A variable of quantity 1 has at most one live use. A variable of quantity 2 may have any number of live uses. The uses in the two arms of a match add up. Until the match fires, both arms are part of the term. If a variable were substituted into both, its value would be copied, and the value may hold calls. This is why the compiler binds `b` inside each arm of `add`. A meet is live code: its two sides are live, and their uses add up.

The quantity is written on the binder. The kind of its type only confirms that the quantity is allowed. That way the live check needs no types.

5.2 Descent

Here is the Ackermann function. Bend accepts it:

```
def ack(+m: Nat, n: Nat) -> Nat:
  match m:
    case 0n:
      1n+n
    case 1n+p:
      match n:
        case 0n:
          ack(p, 1n)
        case 1n+q:
          ack(p, ack(1n+p, q))
```

A live reference to a name k is allowed in two cases. Either k comes earlier in the book than the definition being checked, or k is that definition and its arguments *descend*. To define descent, we read the case tree. Each leading lambda binds a *column*, which is one parameter. A match on a column splits it into *pieces*. A pair match gives its two fields, and a label match records the label it hit. At a self-call, the checker compares the arguments with the columns, from left to right, and skips erased columns. Each argument must be no bigger than its column, until one argument is smaller. The arguments after that one may be anything. The checker compares an argument with a piece of the column's pattern from the top down. The variable bound at the piece, or the label a match hit there, is equal to it. A pair against a piece that was split compares field by field: each field no bigger, and one smaller, makes it smaller; all equal makes it equal. Otherwise the argument is smaller when it is no bigger than another field of the piece. A constructor $(k, (a, (b, ())))$ has the fields a and b; its tag and the pairs that chain the fields are not tried. So `height(Node{rest})` descends under `Node{Cons{x, rest}}`: `rest` is smaller than the field `Cons{x, rest}`.

In `ack`, the two outer calls pass `p`, a piece of `m`, in the first column. The inner call rebuilds `m` as `1n+p` and passes `q`, a piece of `n`, in the second column. The last case uses `p` twice, so `m` must be copyable.

Dead positions have no such rule. A type may refer to any name, as `Nat` and `Nat.arms` do. The body of a definition is live, so the kernel rejects `loop : <> = loop`.

6 Metatheory

This section states the theorems and sketches each proof. All of them are proved in Lean (Section 8). They are about a declarative version of the theory, with three relations:

- *Parallel reduction* $t \Rightarrow u$ reduces any set of redexes in t, in live and dead positions. It includes the rules of Figure 4 with any terms in place of the values, and the unfolding of a name to its body. It also steps $\lambda^2 x.t$ to $\lambda^1 x.t$, and never back: the quantity of a lambda says how it runs, so two lambdas of one liveness are convertible when their bodies are. *Conversion* $a \equiv b$ holds when a and b have a common reduct.
- *Typing* $\Gamma \vdash t : T$ has the rules of Figure 3.
- *Evaluation* $t \mapsto u$ is call-by-value evaluation of live code (Figure 4). Its values are lambdas, matches, labels, rfl, types, pairs of values, and calls that wait for more arguments.

$$
\begin{aligned}
(\lambda^p x.t) \circ^q v &\mapsto t[v/x] \\
\lambda\{(,) : h\} \circ^q (a,^r b) &\mapsto \left(h \circ^{\mathsf{fld}(r,q)} a\right) \circ^q b \\
\lambda\{k : h; m\} \circ^q k &\mapsto h \\
\lambda\{k : h; m\} \circ^q j &\mapsto m \circ^q j \quad (j \neq k) \\
\mathsf{let}^q x = v; t &\mapsto t[v/x] \\
(t : T) &\mapsto t \\
Q_i \sqcap Q_j &\mapsto Q_{\min(i,j)} \\
\mathsf{rfl} \triangleright_{x,h.P} f &\mapsto f \\
k\ v_1 \ldots v_n &\mapsto t[v_1 \ldots v_n]
\end{aligned}
$$

Figure 4. Evaluation of live code. v is a value, and it must be Data when $p = 2$. In the last rule, the values v_i lead through the case tree of k to the leaf t. Evaluation does not enter dead positions.

Evaluation checks two facts at run time that the types promise: a quantity-2 lambda receives a Data value, and a match receives a live argument. Progress shows that typed terms pass these checks. A book is *well typed* when each definition $k : T = t$ has $\vdash T : \mathsf{Type}$ and $\vdash t : T$. It is *live* when each body passes the live check.

6.1 Confluence

Theorem 1 (Confluence). *If* $t \Rightarrow^* u_1$ *and* $t \Rightarrow^* u_2$, *then* $u_1 \Rightarrow^* v$ *and* $u_2 \Rightarrow^* v$ *for some* v.

We follow Takahashi [43]. The complete development t^* reduces every redex of t at once, and steps every λ^2 to λ^1. If $t \Rightarrow u$, then $u \Rightarrow t^*$, and confluence follows. The proof does not use types, so Type : Type plays no part in it.

Confluence gives the two facts about conversion that the rest of the proof needs. First, type formers are injective. If $\forall^p(x : A).B \equiv \forall^q(x : A').B'$, then $p = q$, $A \equiv A'$ and $B \equiv B'$, the same holds for Σ, and $\star\ (g) \equiv \star\ (h)$ only when $g \equiv h$. Second, terms with different head formers are not convertible. So Type is not Data, since Q_1 and Q_2 are distinct labels, and a function type is not an enumeration. Many type theories get these facts from the normalization of types. BendTT gets them from confluence alone.

6.2 Subject Reduction

Theorem 2 (Subject reduction). *If the book is well typed,* $\Gamma \vdash t : T$ *and* $t \Rightarrow u$, *then* $\Gamma \vdash u : T$.

The proof is by induction on the typing derivation, with a substitution lemma and inversion through injectivity. A fit is read the same way: where $U \leq T$ and T converts to a type former, U converts to the same former, and their parts fit. So a β step casts the argument along the fit of the domains, and the result along the fit of the codomains. It holds for reduction anywhere, dead positions included, and it does not need the live check.

Parallel reduction does not preserve the live check. On a closed term, the step from $\lambda^2 x.(x, x)$ to $\lambda^1 x.(x, x)$ leaves a quantity-1 variable used twice. This is harmless, since the live claims are about evaluation, and $\mapsto$ never takes that step. Reduction of open terms loses the live check in a second way. Take $(\lambda^2 x.(x, x))\ y$, where y is an affine variable of type `Nat`. It passes the live check, since it uses y once. One step later it is (y, y), which uses y twice. Quantitative type theory [6, 33] handles this case by scaling the uses of an argument by the quantity of its binder. BendTT counts the argument once and evaluates closed terms only. When a quantity-2 lambda fires in a closed term, its argument is a closed Data value. Its copies hold no variable and no call.

6.3 Progress

Theorem 3 (Progress). *If the book is well typed and live, and $\vdash t : T$ with t closed, then t is a value or $t \mapsto u$ for some u.*

The proof is by induction on the typing derivation, with a canonical forms lemma for each type former. Two side conditions of the typing rules matter here. The rules for matches require a live quantity, so evaluation computes the argument of every match, and a match meets a value. A copyable binder requires a Data type, and a closed value of a Data type holds no lambda and no call.

Lemma 4 (Empty). *No value has the type $\langle\rangle$.*

A value of an enumeration type is one of its labels, and $\langle\rangle$ has none. Other values have other head formers, which are not convertible to an enumeration.

6.4 Termination

Theorem 5 (Termination). *If the book is live, and t is closed and passes the live check, then there is no infinite sequence $t \mapsto t_1 \mapsto t_2 \mapsto \ldots$*

The theorem has no typing hypothesis. The proof gives each term a measure that decreases at every step.

Each live call $k\ a_1 \ldots a_n$ gets a *label*. The label is the index of k in the book, followed by one size per column. The size of a column is the size of the live part of its argument when that argument is a closed value. It is 0 when the column is erased, and ω when the argument is not yet a value. Labels are ordered by index first, and then by sizes, from left to right, with every number below ω. Each lambda, match, let, annotation, rewrite and meet in a live position gets the least label. The *measure* of a term is the multiset of the labels of its live calls and nodes. Multisets are ordered as Dershowitz and Manna define [20]: one element may be replaced by any finite number of smaller ones. This order is well founded because the label order is.

A step that does not fire a call removes a node with the least label. It must not copy anything that has a label, and affinity ensures this. When a quantity-1 lambda fires, its argument goes to at most one live position, so its labels move and are not copied. When a quantity-2 lambda fires, its argument is a Data value, which holds no labels. When a quantity-0 lambda fires, its argument goes to dead positions, which the measure ignores. The substitution may also turn the size ω of some column into a number, which makes a label smaller.

When a call fires, its label is replaced by the labels in the leaf that it reaches. These are calls to earlier definitions, which have a smaller index, and self-calls. The arguments of a self-call are no bigger than their columns, until one is smaller. So the label of the self-call is smaller, whatever its later columns hold. The other labels in the leaf belong to nodes and are the least.

The proof uses one property of affine code: a β step never copies a call. Without affinity, a step could copy a call that has not fired, and the multiset could grow. That is what the paradoxes of Section 3 do.

6.5 Consistency

Theorem 6 (Soundness of the checker). *If the checker accepts a book, the book is well typed and live.*

The checker reduces types by lazy weak-head evaluation, and it compares them one head at a time. Each of its steps is a step of $\Rightarrow^*$, so a conversion that it finds is a conversion of the declarative theory.

Theorem 7 (Consistency). *If the checker accepts a book, then no definition $k : T$ of the book has a type $T \equiv \langle\rangle$.*

Suppose that $k : T$ with $T \equiv \langle\rangle$. By Theorem 6, the book is well typed and live, so $\vdash k : \langle\rangle$, and k passes the live check. By Theorem 5, every evaluation sequence from k is finite, so we follow one to its end. By Theorem 2, every term on the way has type $\langle\rangle$. By Theorem 3, the last term is a value, since it takes no step. By Lemma 4, no value has type $\langle\rangle$.

Every step of this proof is syntactic. In systems such as the calculus of constructions, consistency follows from the normalization of all typed terms, types included. That proof interprets each type as a set of terms, and it cannot work with Type : Type, where some typed terms loop. BendTT gets termination from an untyped measure on live terms, and it uses types only to rule out stuck terms.

Theorem 8 (Adequacy). *If the checker accepts a book, and a closed term t that passes the live check has a type $T \equiv a =_A b$, then $a \equiv b$.*

As in Theorem 7, t evaluates to a value of type $a =_A b$. The only value of an equality type is rfl, which proves only an equality of convertible sides, so $a \equiv b$. So a checked law holds at each closed instance: its proof, applied to closed values, makes the two sides compute to a common term.

7 What Affinity Costs

A closure may be called at most once, even when everything it captures is Data. So the usual `map` is rejected:

```
def map(f: Nat -> Nat, l: List<Nat>) -> List<Nat>:
  match l:
    case Nil{}:
      Nil{}
    case Con{h, t}:
      Con{f(h), map(f, t)}
```

```
Error:
- expected : f
- observed : f (consumed more than once)
Location: map
```

The function `f` is applied once and passed to the recursive call once. There are three ways around this. First, the name of a definition is not a variable, so a program may call a definition any number of times. Second, a *template* parameter, written `~f`, is replaced by its argument at compile time. The base library's `List.map` takes its function this way. A template argument must be closed, so the checker rejects `List.map(~(x => (x + k : U32)), xs)` when `k` is a parameter. Third, the loop can be its own definition, with the captured value as a copyable parameter:

```
def addk(+k: U32, xs: List<U32>) -> List<U32>:
  match xs:
    case Nil{}:
      Nil{}
    case Con{h, t}:
      Con{(h + k : U32), addk(k, t)}
```

The same rule limits impredicative encodings. The Church numeral 2 uses its function twice:

```
def two(-A: Type, f: A -> A, x: A) -> A:
  f(f(x))
```

```
Error:
- expected : f
- observed : f (consumed more than once)
Context:
- A : Type
Location: two
```

So BendTT has impredicative function types, but live code cannot use them to encode data that iterates a function. Bend uses the datatypes of Section 4 instead.

To copy a structure, its type must be Data, and a generic type is Data only when its elements are. The base library declares lists with a quantity parameter:

```
type List<a, -A: Kind(a)> is Kind(a):
  Nil{}
  Con{head: A, tail: List<a, A>}
```

Here `Kind(&1)` is Type and `Kind(&2)` is Data. So `List<&2, Nat>`, written `+List<Nat>`, is Data, and a list of functions is Type. In BendTT, `a` is a variable of type Q, and `List` has the kind $\star\,(a)$. So each generic definition goes to the kernel once, as written. A kind may depend on any value, even one computed at run time.

Negative datatypes are fine, and so is an evaluator that applies each closure once:

```
type Trm is Type:
  Lam{f: Trm -> Trm}
  Num{n: U32}

def app(t: Trm, x: Trm) -> Trm:
  match t:
    case Lam{f}:
      f(x)
    case Num{n}:
      Num{n}
```

A full normalizer for `Trm` must go on with the result of `f(x)`, which is not a piece of `t`. So it needs an extra `Nat` argument that counts down. The recursion rule has two more costs. Mutual recursion is not allowed, so two functions that call each other become one definition with an extra argument that selects which one runs. And a loop whose arguments do not shrink, like the main loop of a server, also needs a counter.

8 Mechanization

The kernel of BendTT, the statements of the theorems and their proofs are one Lean 4 file [38], `bendtt.lean`, of about 4,000 lines. Every proof is complete, and the file declares no axioms of its own, so the proofs rest only on Lean's standard axioms. The file has three parts. Part 1 is the kernel: terms, evaluation, conversion, the checker, the live check, a parser for the text syntax, and a `main` function. Part 2 defines the declarative theory of Section 6 and states the claims of Table 1. Part 3 proves them.

Table 1. The claims of `bendtt.lean`.

Claim	Statement
`confluent`	Theorem 1: parallel reduction is confluent.
`sr`	Theorem 2: parallel reduction preserves types.
`progress`	Theorem 3: a closed typed term is a value or steps.
`empty`	Lemma 4: no value has the empty type.
`halts`	Theorem 5: live evaluation of closed terms ends.
`sound`	Theorem 6: an accepted book is well typed and live.
`consistent`	Theorem 7: no accepted definition has the empty type.
`adequate`	Theorem 8: a closed live proof of $a = b$ has $a \equiv b$.

The theorem `consistent` is about `Book.check`, the function that `main` runs. So the theorem covers the checker that runs, and no model stands between the two. Two parts sit outside the proof. The parser is not verified. And Bend reaches BendTT through a translation, written in TypeScript, that has no proof. The command `bend --verdict` runs this translation and then the kernel. The command `bend -o F.bendtt` writes the translated book to a file, where a reader can check what a law states.

9 Related Work

Type in type. Martin-Löf's first type theory had a type of all types [32], and Girard showed it inconsistent [23]. Coquand studied the paradox in the calculus of constructions [17], and Hurkens simplified it [28]. Howe showed that the paradox

gives a looping term [26], and Meyer and Reinhold studied what it means for type checking [35]. Cardelli proposed Type : Type for a programming language, where the lack of a logic is acceptable [11]. BendTT keeps Type : Type and gets a logic back by restricting the code that runs.

Contraction. In sequent calculus, the rule that uses a hypothesis twice is contraction. Grishin showed that naive set theory is consistent in a logic without contraction [25], and Terui built a naive set theory on light affine logic [44]. Light, elementary and soft logics restrict contraction to bound the cost of normalization [5, 7, 19, 24, 30]. BendTT forbids contraction on functions, allows it on first-order data, and adds structural recursion. It gets termination, but no bound on cost, since the Ackermann function checks.

Quantities. Quantitative type theory [6, 33], used in Idris 2 [10], and graded type theories [2, 37] put quantities on binders, as BendTT does. In these theories the unrestricted quantity is available at every type, and an argument passed to it has its uses scaled. In BendTT the copyable quantity is available only at Data, and an argument passed to it counts once. Linear Haskell adds linear arrows to a language without dependent types [9]. Linear dependent type theories go back to LLF [14] and to Krishnaswami et al. [29].

Two fragments. Zombie [13], from the Trellys project [40], has a logical fragment that is consistent and a programmatic fragment with general recursion. A classifier on each judgment says which fragment a term is in. BendTT also has two parts, with two differences. The position of a term decides its part, so the programmer writes no classifier. And the unrestricted part of BendTT does not run. The partial types of NuPRL also separate total from partial computation [15].

Erasure and impredicativity. The implicit calculus of constructions [36] and Cedille [42] erase parts of terms and have impredicative quantification. Cedille derives induction for λ-encoded data and proves consistency with a semantic model. BendTT builds its datatypes from pairs and labels, affinity limits its encodings, and its consistency proof is syntactic.

Termination and positivity. Coq and Agda require strict positivity [16], since negative types break normalization [34]. Their termination checkers descend from guarded definitions [22] and from the foetus checker [1, 39]. Size-change termination [31] and sized types [4, 8, 27] accept more programs. BendTT's descent check is lexicographic, in the style of foetus. The reason it is sound differs. In Coq and Agda, a termination check is sound because the typed calculus normalizes, and that relies on positivity and universes. In BendTT, the termination argument is untyped, and it relies on affinity.

Verified kernels. MetaCoq [41] and Lean4Lean [12] verify parts of the kernels of large proof assistants, and Abel et al. prove conversion decidable for a type theory in Agda [3]. These projects must handle universe hierarchies and the normalization of types. BendTT has neither.

10 Conclusion

BendTT keeps Type : Type, impredicative function types and negative datatypes, and it is consistent. The code that runs must be affine, copy only first-order data, and recurse on smaller arguments. Curry's and Girard's paradoxes both copy a function, and affinity rejects that copy. Types are exempt from these rules because they never run, and a match cannot look inside a dead value. Termination then follows from a multiset measure on calls that ignores types, and consistency follows from subject reduction and progress.

The main cost is that a closure may be called only once. Several questions are open. BendTT has no semantic model yet. Its equality is intensional. We do not know how far the descent check can be relaxed, for example to size-change termination, while the termination argument stays untyped.

References

[1] Andreas Abel and Thorsten Altenkirch. 2002. A Predicative Analysis of Structural Recursion. *Journal of Functional Programming* 12, 1 (2002), 1–41.

[2] Andreas Abel, Nils Anders Danielsson, and Oskar Eriksson. 2023. A Graded Modal Dependent Type Theory with a Universe and Erasure, Formalized. *Proceedings of the ACM on Programming Languages* 7, ICFP (2023), 220:1–220:35.

[3] Andreas Abel, Joakim Öhman, and Andrea Vezzosi. 2018. Decidability of Conversion for Type Theory in Type Theory. *Proceedings of the ACM on Programming Languages* 2, POPL (2018), 23:1–23:29.

[4] Andreas Abel. 2008. Semi-Continuous Sized Types and Termination. *Logical Methods in Computer Science* 4, 2 (2008).

[5] Andrea Asperti. 1998. Light Affine Logic. In *Proceedings of the 13th Annual IEEE Symposium on Logic in Computer Science (LICS)*, 1998. 300–308.

[6] Robert Atkey. 2018. Syntax and Semantics of Quantitative Type Theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, 2018. 56–65.

[7] Patrick Baillot and Kazushige Terui. 2009. Light Types for Polynomial Time Computation in Lambda Calculus. *Information and Computation* 207, 1 (2009), 41–62.

[8] Gilles Barthe, Maria João Frade, Eduardo Giménez, Luís Pinto, and Tarmo Uustalu. 2004. Type-Based Termination of Recursive Definitions. *Mathematical Structures in Computer Science* 14, 1 (2004), 97–141.

[9] Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. 2018. Linear Haskell: Practical Linearity in a Higher-Order Polymorphic Language. *Proceedings of the ACM on Programming Languages* 2, POPL (2018), 5:1–5:29.

[10] Edwin Brady. 2021. Idris 2: Quantitative Type Theory in Practice. In *35th European Conference on Object-Oriented Programming (ECOOP) (LIPIcs)*, 2021. 9:1–9:26.

[11] Luca Cardelli. 1986. *A Polymorphic Lambda-Calculus with Type:Type.*

[12] Mario Carneiro. 2024. Lean4Lean: Towards a Formalized Metatheory for the Lean Theorem Prover. Retrieved from https://arxiv.org/abs/2403.14064

[13] Chris Casinghino, Vilhelm Sjöberg, and Stephanie Weirich. 2014. Combining Proofs and Programs in a Dependently Typed Language. In *Proceedings of the 41st ACM Symposium on Principles of Programming Languages (POPL)*, 2014. 33–45.

[14] Iliano Cervesato and Frank Pfenning. 2002. A Linear Logical Framework. *Information and Computation* 179, 1 (2002), 19–75.

[15] Robert L. Constable and Scott F. Smith. 1987. Partial Objects in Constructive Type Theory. In *Proceedings of the Second Annual IEEE Symposium on Logic in Computer Science (LICS)*, 1987. 183–193.

[16] Thierry Coquand and Christine Paulin. 1990. Inductively Defined Types. In *COLOG-88: International Conference on Computer Logic (LNCS)*, 1990. Springer, 50–66.

[17] Thierry Coquand. 1986. An Analysis of Girard's Paradox. In *Proceedings of the First Annual IEEE Symposium on Logic in Computer Science (LICS)*, 1986. 227–236.

[18] Haskell B. Curry. 1942. The Inconsistency of Certain Formal Logics. *Journal of Symbolic Logic* 7, 3 (1942), 115–117.

[19] Vincent Danos and Jean-Baptiste Joinet. 2003. Linear Logic and Elementary Time. *Information and Computation* 183, 1 (2003), 123–137.

[20] Nachum Dershowitz and Zohar Manna. 1979. Proving Termination with Multiset Orderings. *Communications of the ACM* 22, 8 (1979), 465–476.

[21] Jana Dunfield and Neel Krishnaswami. 2021. Bidirectional Typing. *ACM Computing Surveys* 54, 5 (2021), 98:1–98:38.

[22] Eduardo Giménez. 1994. Codifying Guarded Definitions with Recursive Schemes. In *Types for Proofs and Programs (TYPES) (LNCS)*, 1994. Springer, 39–59.

[23] Jean-Yves Girard. 1972. Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur. Doctoral dissertation.

[24] Jean-Yves Girard. 1998. Light Linear Logic. *Information and Computation* 143, 2 (1998), 175–204.

[25] V. N. Grišin. 1982. Predicate and Set-Theoretic Calculi Based on Logic Without Contractions. *Mathematics of the USSR–Izvestiya* 18, 1 (1982), 41–59.

[26] Douglas J. Howe. 1987. The Computational Behaviour of Girard's Paradox. In *Proceedings of the Second Annual IEEE Symposium on Logic in Computer Science (LICS)*, 1987. 205–214.

[27] John Hughes, Lars Pareto, and Amr Sabry. 1996. Proving the Correctness of Reactive Systems Using Sized Types. In *Proceedings of the 23rd ACM Symposium on Principles of Programming Languages (POPL)*, 1996. 410–423.

[28] Antonius J. C. Hurkens. 1995. A Simplification of Girard's Paradox. In *Typed Lambda Calculi and Applications (TLCA) (LNCS)*, 1995. Springer, 266–278.

[29] Neelakantan R. Krishnaswami, Pierre Pradic, and Nick Benton. 2015. Integrating Linear and Dependent Types. In *Proceedings of the 42nd ACM Symposium on Principles of Programming Languages (POPL)*, 2015. 17–30.

[30] Yves Lafont. 2004. Soft Linear Logic and Polynomial Time. *Theoretical Computer Science* 318, 1–2 (2004), 163–180.

[31] Chin Soon Lee, Neil D. Jones, and Amir M. Ben-Amram. 2001. The Size-Change Principle for Program Termination. In *Proceedings of the 28th ACM Symposium on Principles of Programming Languages (POPL)*, 2001. 81–92.

[32] Per Martin-Löf. 1971. A Theory of Types.

[33] Conor McBride. 2016. I Got Plenty o' Nuttin'. In *A List of Successes That Can Change the World*. Springer, 207–233.

[34] Nax Paul Mendler. 1991. Inductive Types and Type Constraints in the Second-Order Lambda Calculus. *Annals of Pure and Applied Logic* 51, 1–2 (1991), 159–172.

[35] Albert R. Meyer and Mark B. Reinhold. 1986. "Type" Is Not a Type. In *Proceedings of the 13th ACM Symposium on Principles of Programming Languages (POPL)*, 1986. 287–295.

[36] Alexandre Miquel. 2001. The Implicit Calculus of Constructions. In *Typed Lambda Calculi and Applications (TLCA) (LNCS)*, 2001. Springer, 344–359.

[37] Benjamin Moon, Harley Eades III, and Dominic Orchard. 2021. Graded Modal Dependent Type Theory. In *European Symposium on Programming (ESOP) (LNCS)*, 2021. Springer, 462–490.

[38] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction (CADE 28) (LNCS)*, 2021. Springer, 625–635.

[39] Ulf Norell. 2007. Towards a Practical Programming Language Based on Dependent Type Theory. Doctoral dissertation.

[40] Vilhelm Sjöberg, Chris Casinghino, Ki Yung Ahn, Nathan Collins, Harley D. Eades III, Peng Fu, Garrin Kimmell, Tim Sheard, Aaron Stump, and Stephanie Weirich. 2012. Irrelevance, Heterogeneous Equality, and Call-by-value Dependent Type Systems. *Electronic Proceedings in Theoretical Computer Science (MSFP 2012)* 76, (2012), 112–162.

[41] Matthieu Sozeau, Simon Boulier, Yannick Forster, Nicolas Tabareau, and Théo Winterhalter. 2020. Coq Coq Correct! Verification of Type Checking and Erasure for Coq, in Coq. *Proceedings of the ACM on Programming Languages* 4, POPL (2020), 8:1–8:28.

[42] Aaron Stump. 2017. The Calculus of Dependent Lambda Eliminations. *Journal of Functional Programming* 27, (2017), e14.

[43] Masako Takahashi. 1995. Parallel Reductions in λ-Calculus. *Information and Computation* 118, 1 (1995), 120–127.

[44] Kazushige Terui. 2004. Light Affine Set Theory: A Naive Set Theory of Polynomial Time. *Studia Logica* 77, (2004), 9–40.